14 擴充資料結構 解答

14-E01 select 與 size

推導

x 的左子有 5 個較小元素，x 的整體秩為 6。k=8>6，答案在右子樹，需找右子樹局部第 8-6=2 小。

結論與檢核

因此呼叫 right-select(k'=2)。若 right 為 NIL 而仍查詢，k 已超出 subtree 大小，應報錯。

14-E02 旋轉時 size 更新

推導

旋轉後 x 的子樹包含 x.left、beta，故先算x.size=left.size+beta.size+1。y 的子樹包含 x 與 y.right，再算 y.size=x.size+y.right.size+1。

結論與檢核

必須先 x 後 y；若先算 y 會讀到舊 x.size。其他祖先 size 不因旋轉改變，但若父指標或根位置變更仍要同步。

14-E03 區間重疊查詢

推導

查 [24,26] 先與根重疊，因 20<=26 且 30>=24，可直接回傳根。查 [31,33] 與根不重疊，且 q.low=31>root.high=30，左側即使 max=25 也無法接到 31，但右側 max=34 可能有。

結論與檢核

第二個查詢往右。一般若 x.low>qhigh，候選只能在左；若左.max>=qlow 才需查左。這些條件用於剪枝，不能單靠根區間判定不存在。

14-E04 Select與Rank使用同一棵樹

推導

Select在20時r4、k7，往40改k3；40時r5，往30保留k3；30時r3，回30。Rank27初1，從27上25是right故加1得2；上30、40均從left不加；上20從right加size10+1=4，得6。

結論與檢核

刪20後30少了25卻保留27，size30=3；size40=5；新root25的size=3+5+1=9。不能保留25搬移前的size2；修改要依最終child與後代到祖先順序重算。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 （續）

14-E05 局部公式與全樹Rank的更新成本

推導

新minimum使每個舊node全樹rank加1，若逐node明存精確值就需n次改寫；subtree size只沿新葉的祖先路徑加1，在red-black tree為O(log n)。一般child摘要如height，rotation可能改變局部root摘要而繼續影響祖先。

結論與檢核

子樹內容聚合的size/max在rotation前後整個局部子樹內容相同，新局部root的聚合不變，所以兩個轉軸node重算後祖先不用改；不能把這個額外性質當成所有child公式都具備。

14-E06 Max可行條件不保證有Overlap

推導

left.max40>=5成立，所以搜尋往left[10,40]；但10>6，它完全在query右方，也沒有child可繼續，最後回NIL。left中達max的j即[10,40]，有j.high>=5但不重疊，故j.low10>q.high6。

結論與檢核

所有right的low>=root.low20>=j.low10>6，所以right也不可能重疊；root本身亦不重疊。left.max>=q.low只說不能用『high全太小』排除left，沒有保證left存在解。正確性靠全域low序與不變量。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 （續）

原創驗算程式附錄: augmentation_checks.py

```
"""Augmentation behavior on an ordinary BST; logarithmic height relies on the separate RB
proof."""
from pathlib import Path
import sys
import random
sys.path.insert(0,str(Path(__file__).resolve().parents[2]/"ch12-binary-search-
trees/examples"))
from bst_checks import Tree


def size(node):
    return 0 if node is None else node.size


def maximum(node):
    return float('-inf') if node is None else node.maximum


def low(node):
    return node.key[0] if isinstance(node.key,tuple) else node.key


def pull(node):
    node.size=1+size(node.left)+size(node.right)
    node.maximum=max(node.high,maximum(node.left),maximum(node.right))


def refresh_up(node):
    while node is not None:
        pull(node)
        node=node.parent
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 (續)

```
class AugTree(Tree):
    def insert(self,key,high=None):
        node=super().insert(key)
        node.high=low(node) if high is None else high
        refresh_up(node)
        return node
    def delete(self,z):
        start=z.parent
        if z.left is not None and z.right is not None:
            y=z.right
            while y.left is not None:
                y=y.left
            start=y if y.parent is z else y.parent
        super().delete(z)
        refresh_up(start)
    def rotate_left(self,x):
        y=x.right
        assert y is not None
        p=x.parent
        x.right=y.left
        if y.left is not None:
            y.left.parent=x
        y.parent=p
        if p is None:
            self.root=y
        elif p.left is x:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 （續）

```
            p.left=y
        else:
            p.right=y
        y.left=x
        x.parent=y
        pull(x)
        pull(y)
        return y
    def rotate_right(self,y):
        x=y.left
        assert x is not None
        p=y.parent
        y.left=x.right
        if x.right is not None:
            x.right.parent=y
        x.parent=p
        if p is None:
            self.root=x
        elif p.left is y:
            p.left=x
        else:
            p.right=x
        x.right=y
        y.parent=x
        pull(y)
        pull(x)
        return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 （續）

```
def select(node,k):
    if not 1<=k<=size(node):
        raise ValueError("rank outside subtree")
    while node is not None:
        r=size(node.left)+1
        if k==r:
            return node
        if k<r:
            node=node.left
        else:
            k-=r
            node=node.right


def rank(node):
    result=size(node.left)+1
    while node.parent is not None:
        if node is node.parent.right:
            result+=size(node.parent.left)+1
        node=node.parent
    return result


def overlap_search(tree,a,b):
    node=tree.root
    path=[]
    while node is not None and not (low(node)<=b and a<=node.high):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 (續)

```
        path.append(low(node))
        if node.left is not None and node.left.maximum>=a:
            node=node.left
        else:
            node=node.right
    if node is not None:
        path.append(low(node))
    return node,path


def validate(tree,records):
    seen=set()
    def visit(node,parent,lo,hi):
        if node is None:
            return 0,float('-inf'),[]
        assert node not in seen and node.parent is parent
        assert node.high==records[node.key]
        assert (lo is None or lo<node.key) and (hi is None or node.key<hi)
        seen.add(node)
        ls,lmax,left=visit(node.left,node,lo,node.key)
        rs,rmax,right=visit(node.right,node,node.key,hi)
        assert node.size==ls+rs+1
        assert node.maximum==max(node.high,lmax,rmax)
        return node.size,node.maximum,left+[node.key]+right
    _,_,ordered=visit(tree.root,None,None,None)
    assert ordered==sorted(records)
    for i,key in enumerate(ordered,1):
        node=tree.search(key)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 （續）

```
        assert select(tree.root,i) is node and rank(node)==i


INTERVALS=[(12,17),(5,7),(20,24),(2,4),(8,15),(16,19),(25,27)]


def main():
    tree=AugTree([20,10,40,5,15,30,50,25,35,27])
    assert select(tree.root,7).key==30 and rank(tree.search(27))==6
    tree.delete(tree.search(20))
    assert tree.search(30).size==3 and tree.search(40).size==5 and tree.root.size==9
    rng=random.Random(20261024)
    tree=AugTree()
    records={}
    rotations=0
    for step in range(1500):
        key=rng.randrange(50)
        if rng.randrange(2):
            high=key+rng.randrange(20)
            tree.insert(key,high)
            records[key]=high
        elif key in records:
            tree.delete(tree.search(key))
            del records[key]
        validate(tree,records)
        if records and step%7==0:
            candidates=[tree.search(key) for key in records if tree.search(key).right is not
None]
            if candidates:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 (續)

```
            x=rng.choice(candidates)
            y=tree.rotate_left(x)
            validate(tree,records)
            tree.rotate_right(y)
            validate(tree,records)
            rotations+=2
intervals=AugTree()
for identity,(a,b) in enumerate(INTERVALS):
    intervals.insert((a,identity),b)
assert overlap_search(intervals,9,10)[1]==[12,5,8]
assert overlap_search(intervals,18,18)[1]==[12,20,16]
assert overlap_search(intervals,28,29)==(None,[12,20,25])
assert overlap_search(intervals,4,5)[0] is not None
records={}
tree=AugTree()
queries=0
for step in range(220):
    identity=rng.randrange(16)
    previous=next((key for key in records if key[1]==identity),None)
    if previous is not None:
        tree.delete(tree.search(previous))
        del records[previous]
    if rng.randrange(3):
        a=rng.randrange(25)
        b=a+rng.randrange(8)
        tree.insert((a,identity),b)
        records[(a,identity)]=b
    validate(tree,records)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構 解答 （續）

```
        if records and step%5==0:
            candidates=[tree.search(key) for key in records if tree.search(key).right is not
None]
            if candidates:
                tree.rotate_left(rng.choice(candidates))
                validate(tree,records)
                rotations+=1
        for a in range(-1,33):
            for b in range(a,33):
                found,_=overlap_search(tree,a,b)
                expected=any(key[0]<=b and a<=high for key,high in records.items())
                assert (found is not None)==expected
                if found is not None:
                    assert low(found)<=b and a<=found.high
                queries+=1
    rotation_example=AugTree()
    for a,b in [(12,17),(5,7),(20,24),(8,15),(16,19),(25,27)]:
        rotation_example.insert(a,b)
    x=rotation_example.root
    assert x.maximum==27
    y=rotation_example.rotate_left(x)
    assert x.maximum==19 and y.maximum==27
    print(f"PASS augmented_mixed_updates=1500 actual_rotations={rotations}
rank_select_all_nodes=True interval_queries_against_exhaustive_scan={queries}
closed_endpoints_and_duplicate_lows=True delete_size_trace=(3,5,9) rotated_max=(19,27)")


if __name__=='__main__':
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study